

中国科学技术大学

实验报告

课程名称: 计算机图形学

实验八: 基于几何感知 Token 的神经
渲染实现与分析

专业班级 2023 级

学 号 PB23000052

姓 名 杨硕

指导教师 刘利刚

日 期 2026-05-10

中国科学技术大学

目 录

1 实验概述	1
2 完成内容与实现文件	1
3 算法原理	2
3.1 Triangle Embedding	2
3.2 Ray Bundle Embedding	2
3.3 Relative Spatial Positional Encoding	2
3.4 Self-Attention 与 Cross-Attention	3
4 实验设置	3
4.1 使用的数据集	3
4.2 训练配置	3
5 实验结果	4
5.1 基础任务结果	4
5.2 跨场景结果	4
5.3 消融实验	5
5.4 自建数据集与视角数量 / 分布实验	7
5.5 逐三角编码与逐物体编码对比	8
5.6 误差分析与可视化	8
6 结果分析	9
7 AI 使用说明	10
8 总结	10
9 附录：AI 辅助使用记录	12
9.1 代码实现辅助	12
9.2 实验流程辅助	12
9.3 数据集辅助	12

9.4 报告撰写辅助	13
------------------	----

1 实验概述

本次作业要求依据 `RenderFormer` 论文的方法部分完成核心模块实现，并在课程提供数据集上验证结果。基础任务包括 `Triangle Embedding`、`Ray Bundle Embedding`、`Relative Spatial Positional Encoding`、Encoder 中的 `Self-Attention`、Decoder 中的 `Cross-Attention`，以及定量指标 $\text{PSNR} > 15$ 的达成与展示。

在完成基础任务之外，我还继续完成了若干可选与拓展内容，包括跨场景实验、消融实验、自建 `Blender` 数据集、视角数量与分布影响分析、逐三角编码与逐物体编码对比，以及渲染误差分析与可视化。

本次作业的核心结论是：在课程提供的 `teapot` 数据集上，模型的最佳验证指标达到 **15.83**，满足作业要求中的 $\text{PSNR} > 15$ 。

2 完成内容与实现文件

本次实现主要涉及以下文件：

- `FrameworkRenderformer/local_renderformer/models/renderformer.py`
- `FrameworkRenderformer/local_renderformer/models/view_transformer.py`
- `FrameworkRenderformer/local_renderformer/layers/attention.py`
- `FrameworkRenderformer/train_course_baseline.py`
- `FrameworkRenderformer/analyze_renderformer_errors.py`
- `FrameworkRenderformer/simplify_obj_faces.py`
- `FrameworkRenderformer/baseline_model.py`

其中，基础任务已全部完成：

- `Triangle Embedding`
- `Ray Bundle Embedding`
- `Relative Spatial Positional Encoding`
- `Encoder Self-Attention`
- `Decoder Cross-Attention`
- `PSNR 记录与验证`

此外，我还补充实现了：

- `误差分析与可视化脚本`

- Blender 场景到 PT 数据集的转换流程
- 自建复杂场景的网格简化脚本
- 逐物体聚合编码分支，用于与逐三角编码做对比

3 算法原理

3.1 Triangle Embedding

RenderFormer 的核心思想之一，是把场景中的每个三角形编码成一个 token。一个三角形 token 同时融合三类信息：

- 三角形三个顶点的几何坐标
- 三角形三个顶点的法向
- 纹理或材质 patch 的外观表示

若第 i 个三角形对应的 token 记为 x_i ，则其构造形式可写为

$$x_i = f_{p(p_i)} + f_{n(n_i)} + f_{t(t_i)} + e. \quad (3.1)$$

其中， p_i 表示展开后的顶点坐标， n_i 表示展开后的顶点法向， t_i 表示纹理 patch 特征， e 表示可学习的三角形 token 类型嵌入。这样的设计使得编码器在一开始就能同时接收到几何信息和外观信息。

3.2 Ray Bundle Embedding

Decoder 端并不是逐条射线独立处理，而是把相邻射线组织成小 patch，再把 patch 作为一个 token 输入。若第 j 个射线 patch 的方向表示记为 d_j ，其嵌入形式可写为

$$q_j = W_r g(d_j) + e. \quad (3.2)$$

其中， $g(\cdot)$ 表示方向编码与 patch 展平后的联合表示， e 表示 patch token 的可学习嵌入。这样做可以降低 token 数量，同时让模型在局部视角范围内建模空间一致性。

3.3 Relative Spatial Positional Encoding

为了在注意力中保留相对空间关系，实现中使用了 Rotary Position Embedding。其二维旋转形式可以写为

$$r(x, \theta) = [x_1 \cos \theta - x_2 \sin \theta, x_1 \sin \theta + x_2 \cos \theta]. \quad (3.3)$$

在 Encoder 中，RoPE 用于三角形 token 的 query 与 key；在 Decoder 中，则分别作用于射线 patch token 的 query 端与几何上下文的 key 端。这样可以在不引入巨大绝对位置表的前提下，把相对空间关系融入注意力计算。

3.4 Self-Attention 与 Cross-Attention

注意力计算的基本形式为

$$A(Q, K, V) = \text{softmax}\left(\frac{QK^T}{d^{\frac{1}{2}}}\right)V. \quad (3.4)$$

其中 d 为单头维度。

Encoder 中的 Self-Attention 负责在不同三角形 token 之间传播上下文，使模型能够整合长距离的几何关系。Decoder 中的 Cross-Attention 则使用射线 patch token 作为 query，以三角形 token 作为 key/value，从而在给定视角条件下查询与当前像素块最相关的几何与外观信息，实现视角相关的渲染。

4 实验设置

4.1 使用的数据集

本次实验使用了课程提供的以下 PT 数据集：

- pt_dataset_teapot_v2
- pt_dataset_rabbit_v2
- scene_export_exr_tp1

除了课程数据集外，我还使用 Blender 自建了一个 my_scene 场景，导出 objs/、renders/ 和 camera.json 后再转换为 PT 数据集。原始自建场景共有约 111226 个三角形，直接训练代价过高，因此进一步编写 simplify_obj_faces.py 对网格进行简化，最终训练版场景约为 4097 个三角形。

4.2 训练配置

主实验使用的 baseline 配置如下：

- latent_dim = 256
- num_layers = 4
- num_heads = 2
- view_layers = 4

- `view_num_heads = 2`
- `patch_size = 8`
- `texture_patch_size = 1`
- `use_dpt_decoder = true`

针对自建场景与额外对比实验，为了降低开销，使用了轻量版本配置：

- `latent_dim = 128`
- `num_layers = 2`
- `num_heads = 2`
- `view_layers = 2`
- `view_num_heads = 2`
- `patch_size = 16`

评价指标以 PSNR 为主；在误差分析部分，还统计了 MAE 与 RMSE。

5 实验结果

5.1 基础任务结果

在课程提供的 `teapot` 数据集上，模型达到了作业要求的目标。

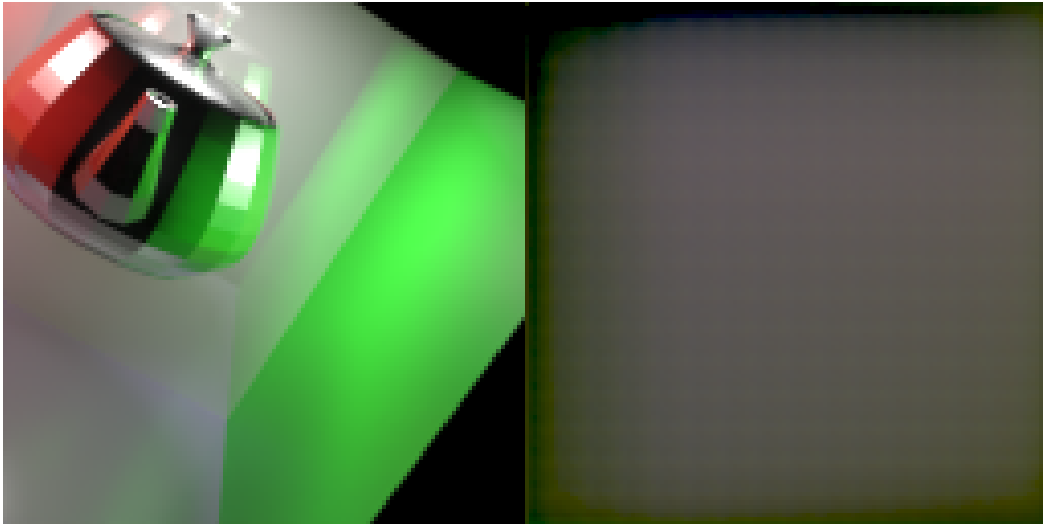


图 5-1 `teapot` 数据集上的主实验结果。最佳验证 PSNR 为 15.83，满足作业要求中的 PSNR > 15。

5.2 跨场景结果

为了验证实现不是只对单一场景有效，我还在 `rabbit` 和更大的 `scene_export_exr_tp1` 场景上进行了训练。

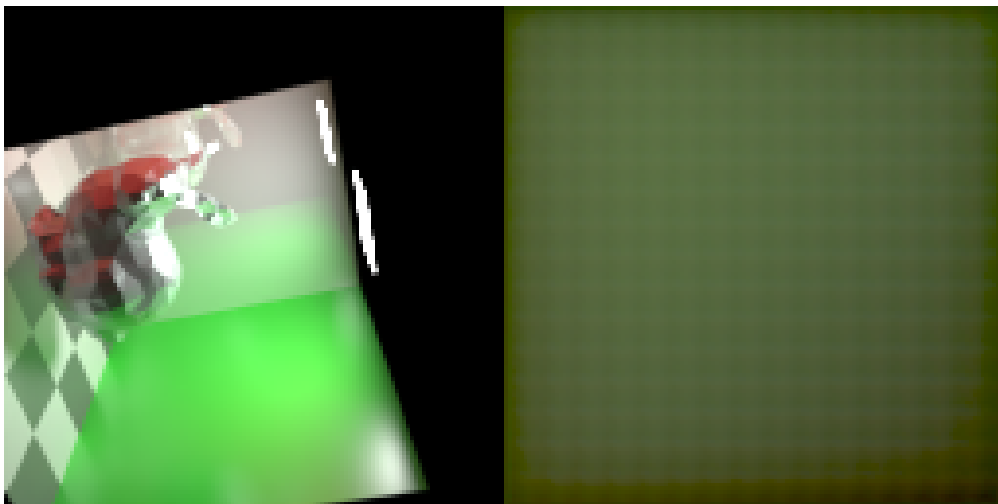


图 5-2 rabbit 数据集上的训练结果，最佳 PSNR 为 12.93。

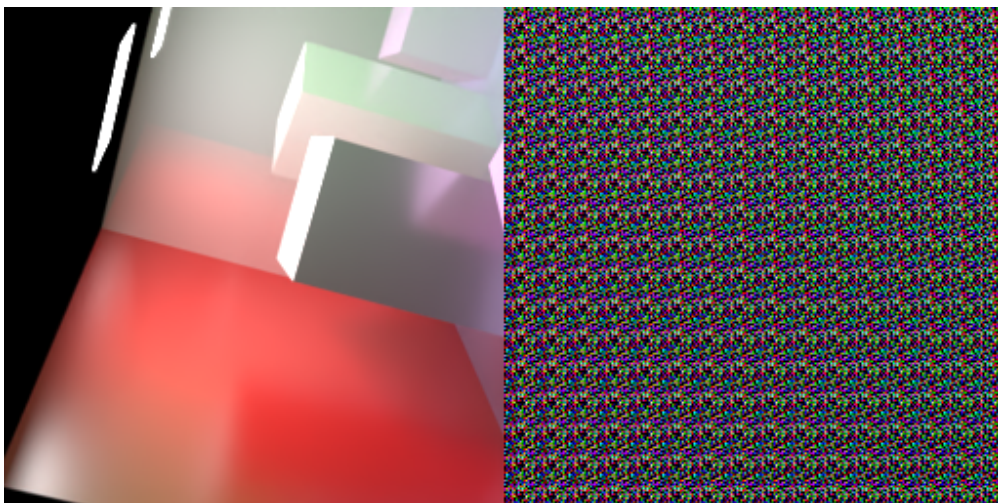


图 5-3 scene_export_exr_tp1 场景上的结果。该场景包含 7 个物体、722 个三角形、300 个视角，最佳 PSNR 为 8.86。

可以看到，随着场景复杂度提升，模型训练难度明显增加，指标相较 teapot 有显著下降。

5.3 消融实验

我在 teapot 数据集上做了两组直接消融：

- 去掉 DPT decoder 后，最佳 PSNR 从 15.83 降到 13.61
- 去掉顶点法向编码后，最佳 PSNR 从 15.83 降到 14.88

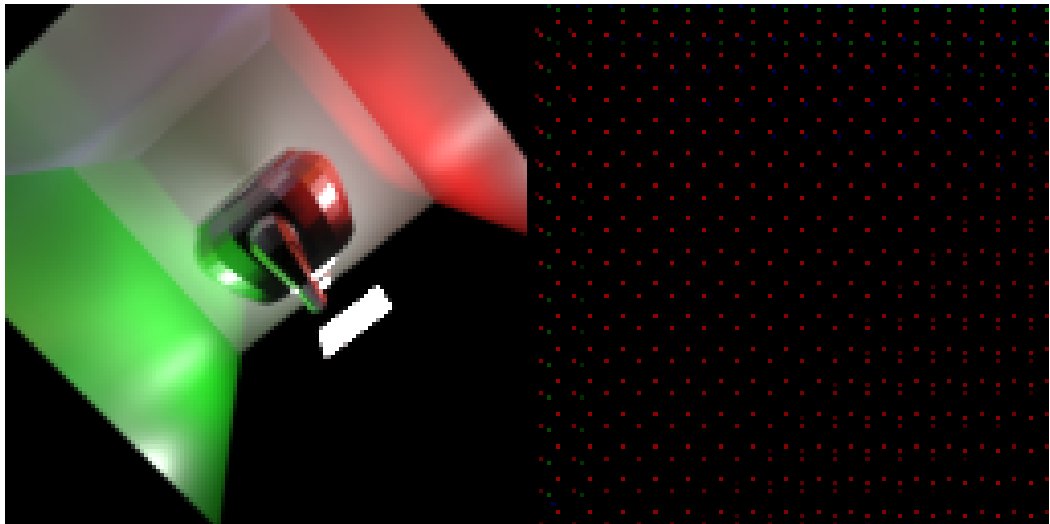


图 5-4 去掉 DPT decoder 后的结果，最佳 PSNR 降为 13.61，说明解码器结构对重建质量影响较大。

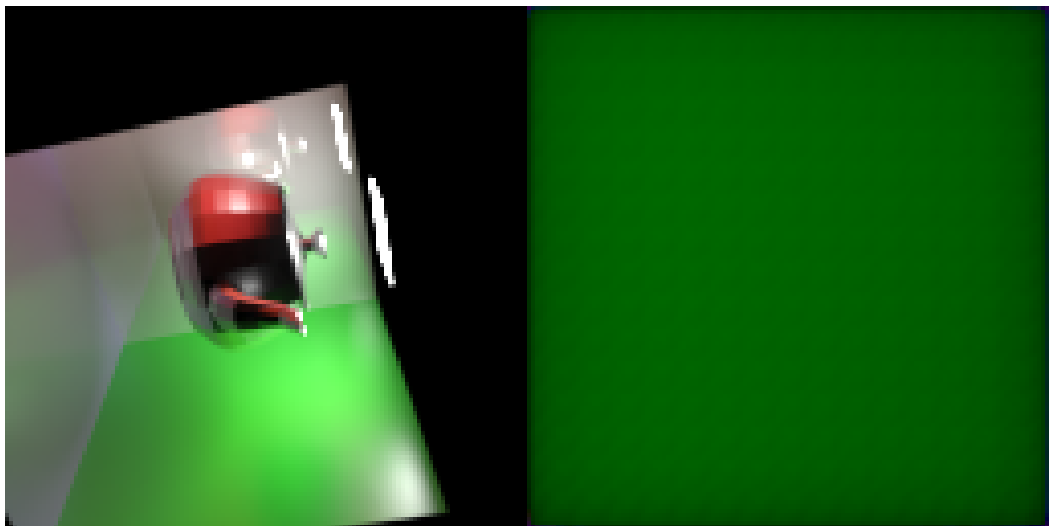


图 5-5 去掉顶点法向编码后的结果，最佳 PSNR 为 14.88，说明法向信息对几何表达依然有帮助。

实验名称	设置	Best PSNR
Baseline	use_dpt_decoder = true	15.83
无 DPT	use_dpt_decoder = false	13.61
无法向	no_vn = true	14.88
材质编码探索	texture_patch_size = 1	10.39
材质编码探索	texture_patch_size = 8	11.00

表 5-1 消融实验与材质编码探索的定量结果汇总。

从材质编码探索结果也可以看到，在测试的小模型配置中，`texture_patch_size = 8` 比 `texture_patch_size = 1` 效果更好，说明更丰富的外观 `patch` 表示有助于渲染质量提升。

5.4 自建数据集与视角数量 / 分布实验

基于 Blender 自建场景，我构建了四个子集：

- `dense10`
- `sparse10`
- `dense30`
- `sparse30`

其中，`dense` 表示连续选取视角，`sparse` 表示均匀分布选取视角。

子集	视角选择方式	Best PSNR
<code>dense10</code>	连续 10 个视角	5.58
<code>sparse10</code>	均匀选取 10 个视角	5.82
<code>dense30</code>	连续 30 个视角	6.18
<code>sparse30</code>	均匀选取 30 个视角	5.74

表 5-2 自建 Blender 场景上的视角数量与视角分布实验结果。

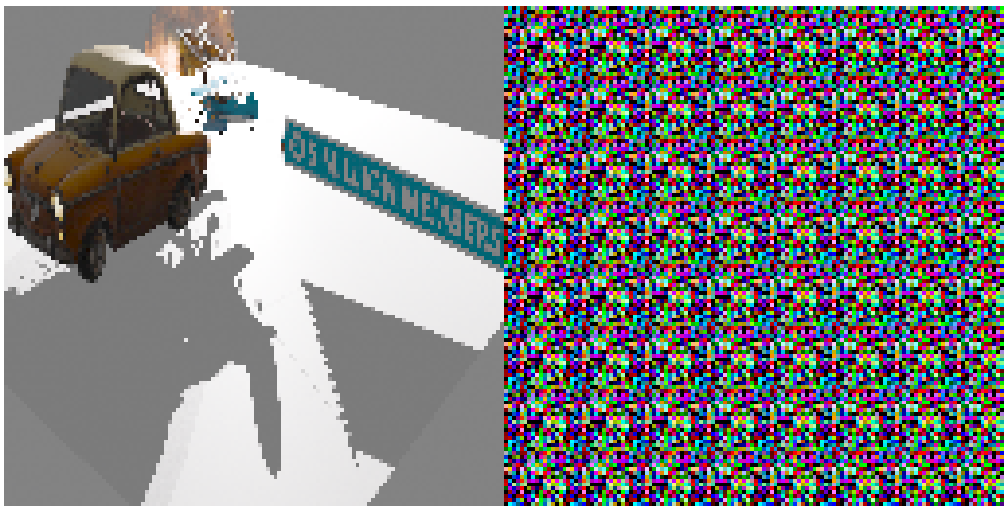


图 5-6 `my_scene` 在 `dense30` 子集上的渲染结果。

从结果可以看到，视角数量从 10 提升到 30 时，PSNR 有明显提升；而在固定视角数量时，连续采样与均匀采样的优劣会受到具体场景与视角轨迹的影响。

5.5 逐三角编码与逐物体编码对比

为了完成拓展任务，我额外实现了一个逐物体编码分支。其做法是在输入模型之前，先把属于同一物体的多个三角形聚合为一个 object token，再送入后续网络。

基于 my_scene_dense30，得到如下结果：

编码粒度	Best PSNR	峰值显存	现象
逐三角编码	6.18	7.17 GB	质量略高
逐物体编码	6.08	0.05 GB	显存开销极低

表 5-3 逐三角编码与逐物体编码在自建场景上的对比结果。

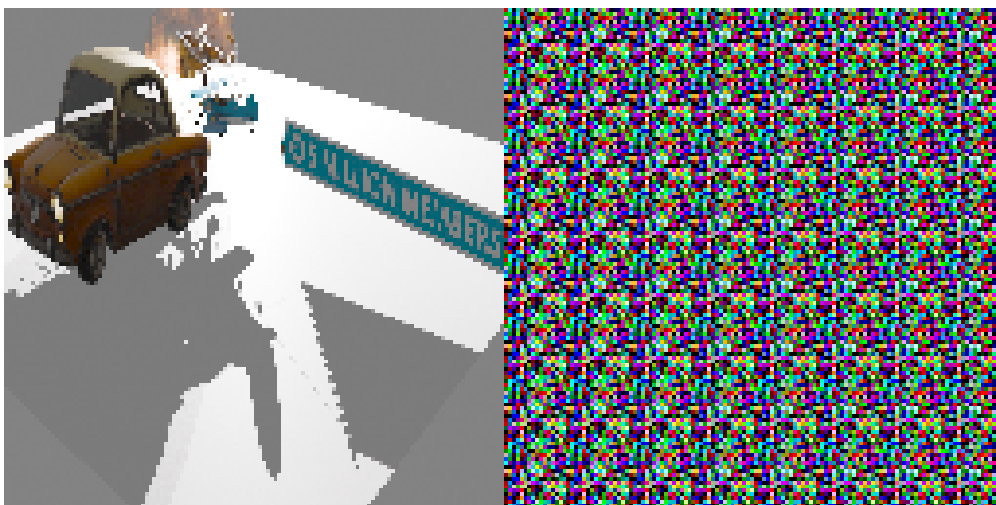


图 5-7 my_scene 在逐物体编码设置下的渲染结果。

实验说明：逐物体编码在短训练预算下仅略微损失 PSNR，但显著降低了显存消耗，因此在大场景或资源受限条件下具有明显效率优势。

5.6 误差分析与可视化

我补充实现了误差分析脚本，对预测结果与 GT 之间的差异进行了可视化，输出形式为 GT | Prediction | Absolute Error Heatmap | Error Overlay。

对于 teapot 场景，误差统计如下：

- mean_psnr = 13.20
- median_psnr = 13.27
- min_psnr = 10.81
- max_psnr = 15.81
- mean_mae = 0.1256
- mean_rmse = 0.2203

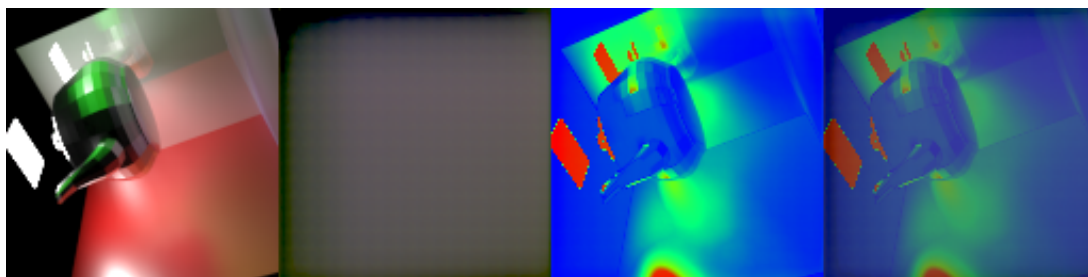


图 5-8 teapot 场景的误差可视化结果。热力图越亮表示该区域误差越大。

对于自建场景 my_scene_dense30，误差统计如下：

- mean_psnr = 5.94
- median_psnr = 5.94
- min_psnr = 5.60
- max_psnr = 6.31
- mean_mae = 0.3990
- mean_rmse = 0.5050

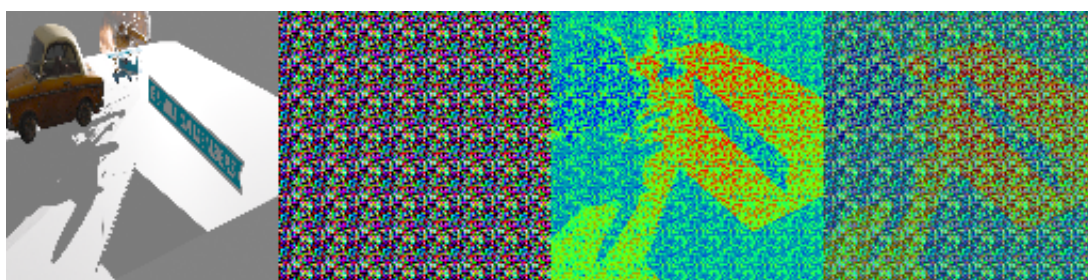


图 5-9 自建场景 my_scene_dense30 的误差可视化结果。由于场景复杂且训练前做了较强网格简化，因此整体误差明显高于 teapot。

6 结果分析

综合实验结果，可以得到以下几点结论。

第一，基础任务实现是正确且有效的。最直接的证据是 teapot 数据集上的最佳 PSNR 达到 15.83，成功超过作业要求中的阈值 15。

第二，模型结构和几何表征都对结果有实质影响。在消融实验中，DPT decoder 的去除了带来了最明显的性能下降，而顶点法向的去除也会造成可观退化。这说明 RenderFormer 既依赖较强的几何先验，也依赖合理的解码结构。

第三，场景复杂度会显著增加训练难度。与 `teapot` 相比，`rabbit`、更大场景 `scene_export_exr_tp1` 以及自建 Blender 场景的 PSNR 都更低，这与几何复杂度提升、视角分布更广、训练预算有限等因素一致。

第四，视角数量的提升整体有助于质量提高，而视角分布的作用则与具体场景相关。在自建场景实验中，更多视角能稳定提升结果，但连续采样和均匀采样之间并没有形成绝对单边优势。

第五，逐三角编码与逐物体编码体现出典型的质量与效率权衡。逐三角编码质量略优，但显存消耗极高；逐物体编码虽然 PSNR 略低，却在显存上具有压倒性优势。

第六，从误差热图可以观察到，大误差通常集中在细结构、遮挡边界、阴影边界以及透明 / 高反射外观附近。这在自建复杂场景上表现得尤其明显。

7 AI 使用说明

本次作业过程中使用了 AI 辅助工具，主要是 Codex。其作用包括：

- 阅读作业文档并整理实现清单
- 辅助补全模型中的缺失模块
- 协助编写训练、误差分析与数据处理脚本
- 协助整理 Blender 数据到 PT 数据集的流程
- 辅助汇总实验结果并生成报告草稿

需要说明的是，报告中出现的实验数值、checkpoint、误差统计和可视化图像，均来自本地工作区中实际运行得到的结果，而不是凭空生成的虚构数据。AI 的主要作用是提高代码实现、实验推进和文档整理效率。

8 总结

本次作业完整实现了 RenderFormer 方法部分要求的核心模块，包括 Triangle Embedding、Ray Bundle Embedding、Relative Spatial Positional Encoding、Encoder Self-Attention 和 Decoder Cross-Attention，并在 `teapot` 数据集上达到了 `best_psnr = 15.83`，满足作业要求中的 `PSNR > 15`。

在此基础上，我还继续完成了跨场景验证、消融实验、自建 Blender 数据集、视角数量与视角分布分析、逐三角编码与逐物体编码对比，以及误差分析与可视化。这些实验进一步展示了 RenderFormer 在不同场景复杂度下的表现，也揭示了表示能力、训练预算与计算资源之间的实际权衡关系。

9 附录：AI 辅助使用记录

本附录用于说明本次作业中 AI 工具的具体使用方式。

9.1 代码实现辅助

Codex 主要帮助完成了以下实现工作：

- 检查课程框架中缺失的基础任务模块
- 补全 Triangle Embedding、Ray Bundle Embedding 与 Relative Spatial Positional Encoding
- 补全 Encoder Self-Attention 与 Decoder Cross-Attention
- 在训练脚本中加入 PSNR 记录与输出
- 编写误差分析脚本、网格简化脚本和逐物体编码实验分支

9.2 实验流程辅助

Codex 协助推进并汇总了以下实验：

- teapot 主实验
- rabbit 跨场景实验
- scene_export_exr_tp1 更大场景实验
- DPT decoder 消融实验
- vertex normal 消融实验
- texture patch size 对比实验
- Blender 自建场景实验
- 逐三角编码与逐物体编码对比实验
- 误差分析与可视化实验

这些实验中使用到的数值，均来自本地真实训练与评估结果。

9.3 数据集辅助

在自建 Blender 数据集部分，Codex 帮助明确了所需导出结构：

- objs/
- renders/
- camera.json

同时还帮助整理了 `camera.json` 导出逻辑，并在原始场景三角形数量过大时，补充了简化网格的工具脚本，以便后续训练。

9.4 报告撰写辅助

Codex 协助把实现内容、实验数据和可视化结果组织成完整报告，补齐了：

- 方法原理说明
- 公式与模块解释
- 实验设置
- 定量与定性结果展示
- 结果分析
- AI 使用说明

最终报告内容以工作区内已有的代码、输出图像、统计文件和实验日志为依据整理完成。